CASE STUDY 1: Electricity Bill Calculator

1. Problem Statement

Develop a Python program to calculate the electricity bill based on the number of units consumed. Apply different charges for different unit ranges and add a fixed charge.

2. Objectives

- Calculate electricity charges.
- Apply slab-wise billing.
- Calculate the final bill.
- Identify invalid unit values.

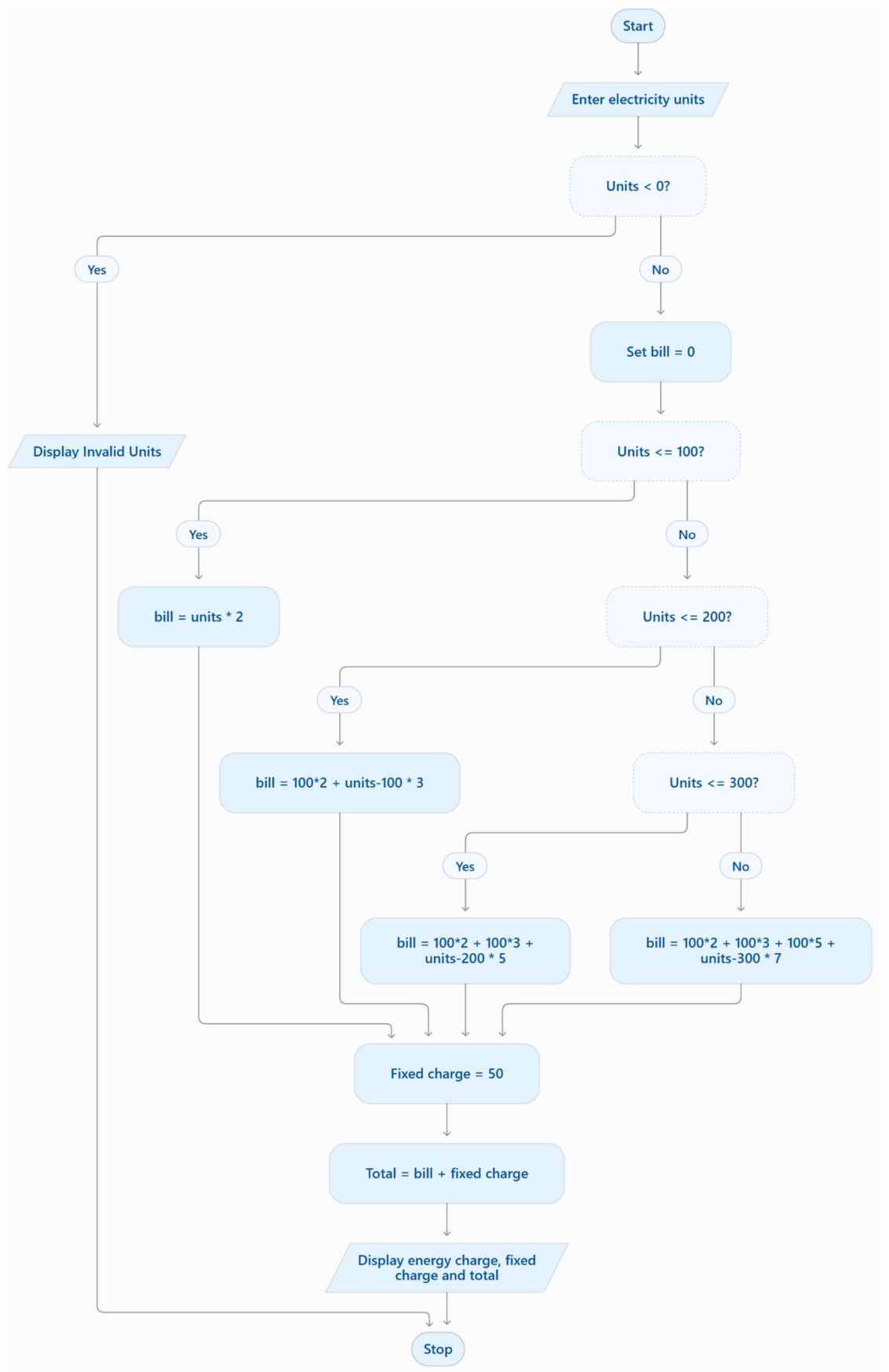
3. Assumptions

- First 100 units: ₹2 per unit.
- Next 100 units: ₹3 per unit.
- Next 100 units: ₹5 per unit.
- Above 300 units: ₹7 per unit.
- Fixed charge: ₹50.
- Negative units are invalid.

4. Algorithm

1. Start.
2. Read electricity units.
3. Check whether units are negative.
4. If negative, display invalid units.
5. Otherwise, calculate the bill using slab rates.
6. Add fixed charge.
7. Display energy charge and total bill.
8. Stop.

5.flowchart


Start
Enter electricity units
Units < 0?
Yes
No
Set bill = 0
Display Invalid Units
Units <= 100?
Yes
No
bill = units * 2
Units <= 200?
Yes
No
bill = 100*2 + units-100 * 3
Units <= 300?
Yes
No
bill = 100*2 + 100*3 + units-200 * 5
bill = 100*2 + 100*3 + 100*5 + units-300 * 7
Fixed charge = 50
Total = bill + fixed charge
Display energy charge, fixed charge and total
Stop

6. Pseudocode

```
START
INPUT units
IF units < 0
   PRINT Invalid units
ELSE
   IF units <= 100
      bill = units * 2
   ELSE IF units <= 200
      bill = 200 + (units - 100) * 3
   ELSE IF units <= 300
      bill = 500 + (units - 200) * 5
   ELSE
      bill = 1000 + (units - 300) * 7
   END IF
   total = bill + 50
   PRINT bill, total
END IF
STOP
```

7. Python Program

```
      units = int(input("Enter electricity units: "))


if units < 0:
   print("Invalid units")
else:
   if units <= 100:
      bill = units * 2
   elif units <= 200:
```

```
    bill = 100 * 2 + (units - 100) * 3
elif units <= 300:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
else:
    bill = 100 * 2 + 100 * 3 + 100 * 5 + (units - 300) * 7

fixed_charge = 50
total = bill + fixed_charge

print("Energy charge:", bill)
print("Fixed charge:", fixed_charge)
print("Total electricity bill:", total)
```

9. Test Cases

Units	Expected total
50	₹150
150	₹400
250	₹800
350	₹1,400
-10	Invalid units

9. Conclusion

This program calculates electricity bills using conditional statements and slab-wise rates. It helps understand real-life applications of decision-making statements in Python.

CASE STUDY 2: Employee Salary Calculator

1. Problem Statement

Develop a Python program to calculate an employee's net salary based on basic salary, allowances, overtime and incentives. Apply deductions according to gross salary.

2. Objectives

- Calculate gross salary.
- Apply percentage-based deductions.
- Calculate net salary.
- Display the salary statement.

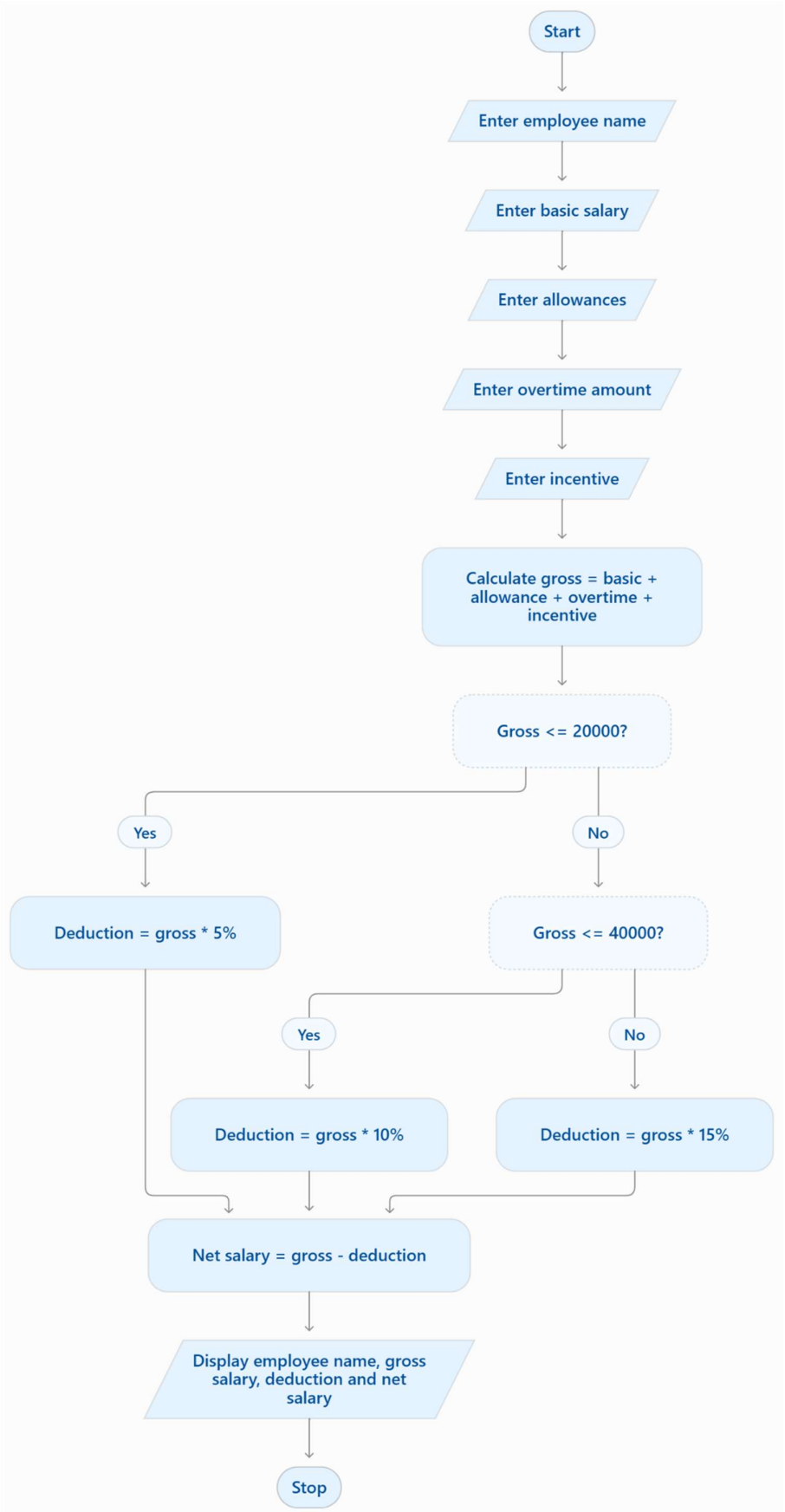
3. Assumptions

- Gross salary = Basic salary + Allowances + Overtime + Incentives.
- Gross salary up to ₹20,000: 5% deduction.
- ₹20,001–₹40,000: 10% deduction.
- Above ₹40,000: 15% deduction.
- All salary inputs are non-negative.

4. Algorithm

1. Start.
2. Read employee name and salary details.
3. Calculate gross salary.
4. Check the gross salary range.
5. Calculate deduction.
6. Calculate net salary.
7. Display salary statement.
8. Stop.

5. Flowchart


Start
Enter employee name
Enter basic salary
Enter allowances
Enter overtime amount
Enter incentive
Calculate gross = basic + allowance + overtime + incentive
Gross <= 20000?
Yes
No
Deduction = gross * 5%
Gross <= 40000?
Yes
No
Deduction = gross * 10%
Deduction = gross * 15%
Net salary = gross - deduction
Display employee name, gross salary, deduction and net salary
Stop

6. Pseudocode

```
START
INPUT name, basic, allowance, overtime, incentive
gross = basic + allowance + overtime + incentive

IF gross <= 20000
   deduction = gross * 0.05
ELSE IF gross <= 40000
   deduction = gross * 0.10
ELSE
   deduction = gross * 0.15
END IF

net = gross - deduction
PRINT gross, deduction, net
STOP
```

7. Python Program

```
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: "))
allowance = float(input("Enter allowances: "))
overtime = float(input("Enter overtime amount: "))
incentive = float(input("Enter incentive: "))

gross = basic + allowance + overtime + incentive

if gross <= 20000:
   deduction = gross * 0.05
elif gross <= 40000:
```

```
    deduction = gross * 0.10
else:
    deduction = gross * 0.15

net_salary = gross - deduction

print("\n--- Salary Statement ---")
print("Employee:", name)
print("Gross salary:", round(gross, 2))
print("Deduction:", round(deduction, 2))
print("Net salary:", round(net_salary, 2))
```

8. Test Cases

Gross salary	Deduction	Net salary
₹15,000	₹750	₹14,250
₹30,000	₹3,000	₹27,000
₹50,000	₹7,500	₹42,500

9. Conclusion

This program demonstrates arithmetic operations and conditional statements to calculate employee salaries.

CASE STUDY 3: Product Billing System

1. Problem Statement

Develop a Python program to generate a product bill by accepting product names, prices and quantities. Calculate discounts based on product amounts and display the final bill.

2. Objectives

- Accept multiple products.

- Calculate individual product amounts.
- Apply discounts.
- Calculate the total bill.

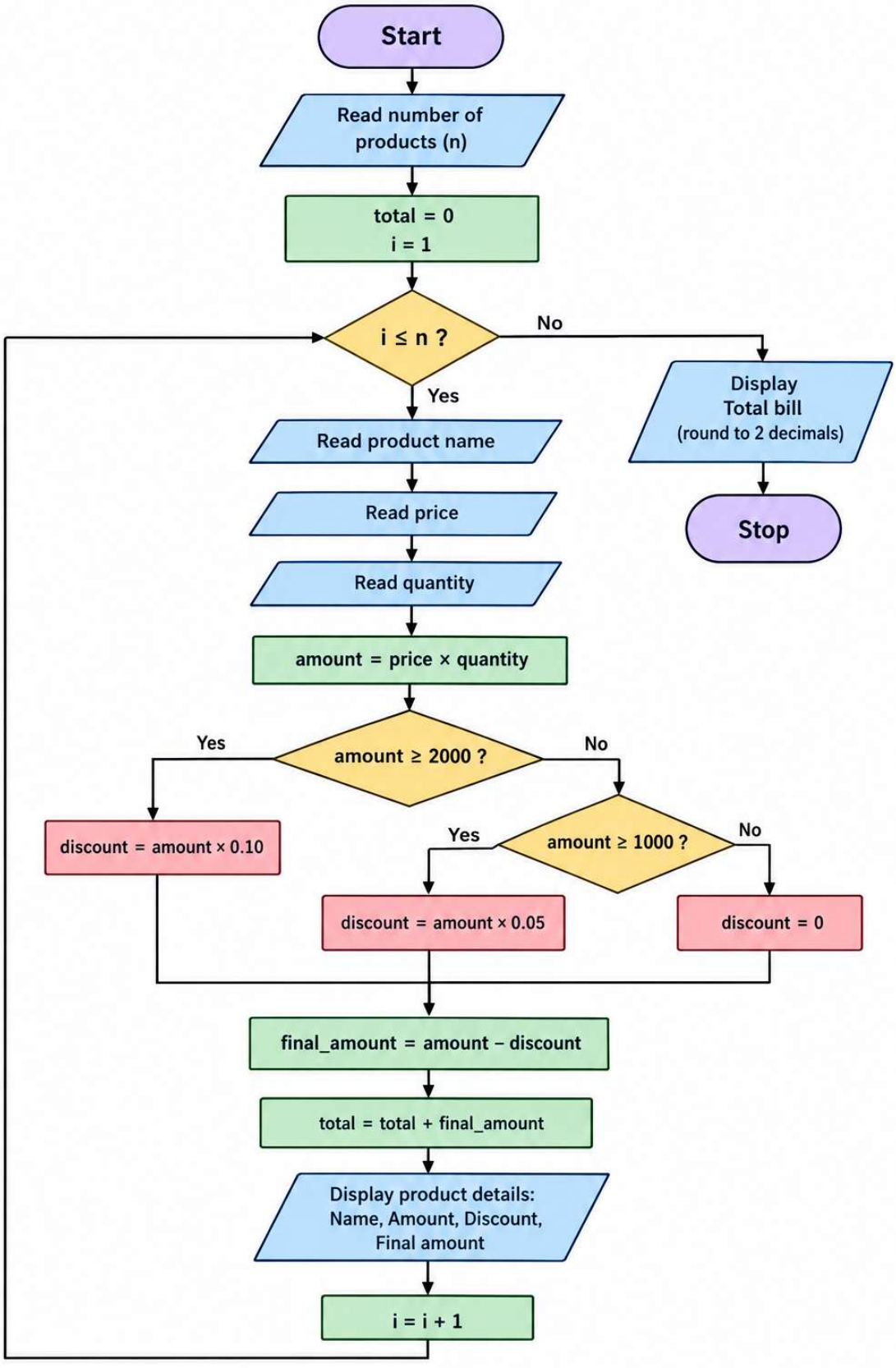
3. Assumptions

- Amount = Price × Quantity.
- Amount ₹2,000 or above: 10% discount.
- Amount ₹1,000–₹1,999.99: 5% discount.
- Below ₹1,000: No discount.
- Product price and quantity must be non-negative and positive respectively.

4. Algorithm

1. Start.
2. Read the number of products.
3. Initialize total to zero.
4. Repeat for each product.
5. Read product name, price and quantity.
6. Calculate amount.
7. Calculate discount.
8. Calculate final amount.
9. Add final amount to total.
10. Display the total bill.
11. Stop.

5. Flowchart


Start
Read number of products (n)
total = 0
i = 1
i ≤ n ?
No
Yes
Display Total bill (round to 2 decimals)
Stop
Read product name
Read price
Read quantity
amount = price × quantity
amount ≥ 2000 ?
Yes
No
discount = amount × 0.10
amount ≥ 1000 ?
Yes
No
discount = amount × 0.05
discount = 0
final_amount = amount − discount
total = total + final_amount
Display product details: Name, Amount, Discount, Final amount
i = i + 1

6. Pseudocode

```
START
INPUT n
total = 0

FOR each product
  INPUT name, price, quantity
  amount = price * quantity

  IF amount >= 2000
    discount = amount * 0.10
  ELSE IF amount >= 1000
    discount = amount * 0.05
  ELSE
    discount = 0
  END IF

  final = amount - discount
  total = total + final
  PRINT final
END FOR

PRINT total
STOP
```

7. Python Program

```
n = int(input("Enter number of products: "))

total = 0
```

```
for i in range(n):
    name = input("Enter product name: ")
    price = float(input("Enter price: "))
    quantity = int(input("Enter quantity: "))

    amount = price * quantity

    if amount >= 2000:
        discount = amount * 0.10
    elif amount >= 1000:
        discount = amount * 0.05
    else:
        discount = 0

    final_amount = amount - discount
    total += final_amount

    print("\nProduct:", name)
    print("Amount:", round(amount, 2))
    print("Discount:", round(discount, 2))
    print("Final amount:", round(final_amount, 2))

print("\nTotal bill:", round(total, 2))
```

8. Test Cases

Product	Price	Quantity	Final amount
Bag	₹2,500	1	₹2,250

Book	₹500	2	₹950
Pen	₹100	2	₹200

Expected total bill: ₹3,400

9. Conclusion

This program demonstrates the use of loops, conditional statements and arithmetic operations in a real-life shopping application.

CASE STUDY 4: Student Attendance Calculator

1. Problem Statement

Develop a Python program to calculate a student's attendance percentage and determine how many additional classes the student must attend to reach the required attendance percentage.

2. Objectives

- Calculate attendance percentage.
- Check attendance eligibility.
- Find additional classes required.
- Display attendance recovery percentage.

3. Assumptions

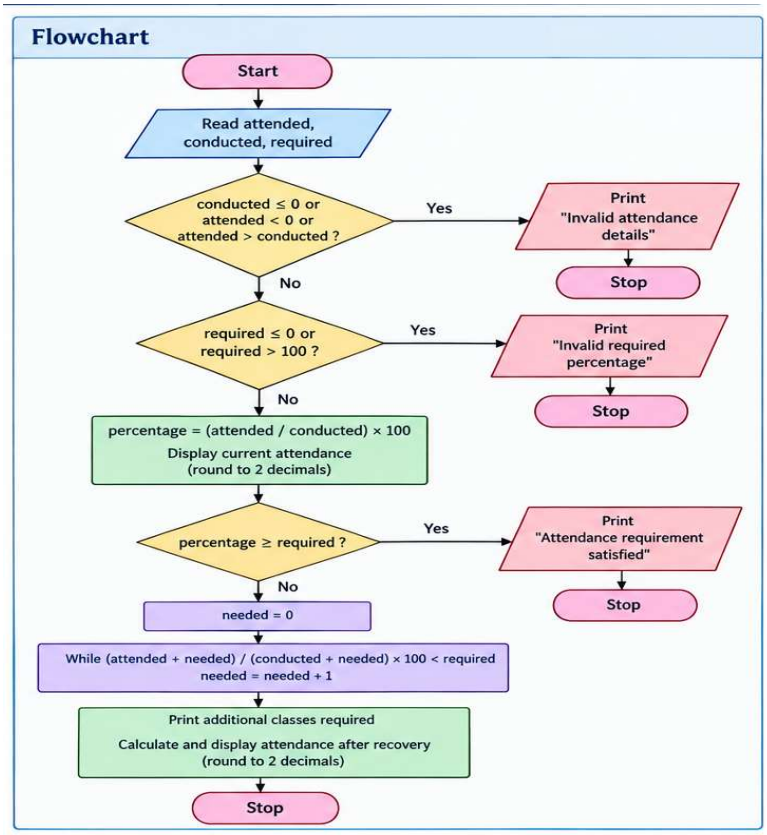
- Total classes conducted must be positive.
- Attended classes cannot exceed conducted classes.
- Required attendance percentage must be between 0 and 100.

4. Algorithm

1. Start.
2. Input attended and conducted classes.
3. Input required attendance percentage.
4. Validate the inputs.
5. Calculate current attendance percentage.

6. Check whether the requirement is satisfied.
7. If not, use a while loop to calculate additional classes.
8. Display the result.
9. Stop.

5.flowchart:



6. Pseudocode

START

INPUT attended, conducted, required

VALIDATE inputs

```
percentage = attended / conducted * 100

IF percentage >= required
   PRINT Requirement satisfied
ELSE
   needed = 0
   WHILE (attended + needed) /
        (conducted + needed) * 100 < required
      needed = needed + 1
   END WHILE
   PRINT needed
END IF
STOP
```

7. Python Program

```
attended = int(input("Enter classes attended: "))
conducted = int(input("Enter total classes conducted: "))
required = float(input("Enter required attendance percentage: "))

if conducted <= 0 or attended < 0 or attended > conducted:
   print("Invalid attendance details")
elif required <= 0 or required > 100:
   print("Invalid required percentage")
else:
   percentage = attended / conducted * 100

   print("Current attendance:", round(percentage, 2), "%")

   if percentage >= required:
```

```
    print("Attendance requirement satisfied")
else:
    needed = 0

    while ((attended + needed) /
        (conducted + needed) * 100) < required:
        needed += 1

    print("Additional classes required:", needed)
    print("Attendance after recovery:",
        round((attended + needed) /
            (conducted + needed) * 100, 2), "%")
```

8. Test Cases

Attended	Conducted	Required	Expected result
80	100	75%	Satisfied
60	100	75%	60 additional classes
70	100	75%	20 additional classes
110	100	75%	Invalid details

9. Conclusion

This program demonstrates the use of while loops and conditional statements to solve a real-life student attendance problem.

CASE STUDY 5: Bank Transaction Management

1. Problem Statement

Develop a Python program to manage bank transactions by allowing users to deposit and withdraw money. Calculate total deposits, total withdrawals and the final balance.

2. Objectives

- Perform deposits and withdrawals.
- Maintain account balance.
- Prevent invalid withdrawals.
- Display transaction summary.

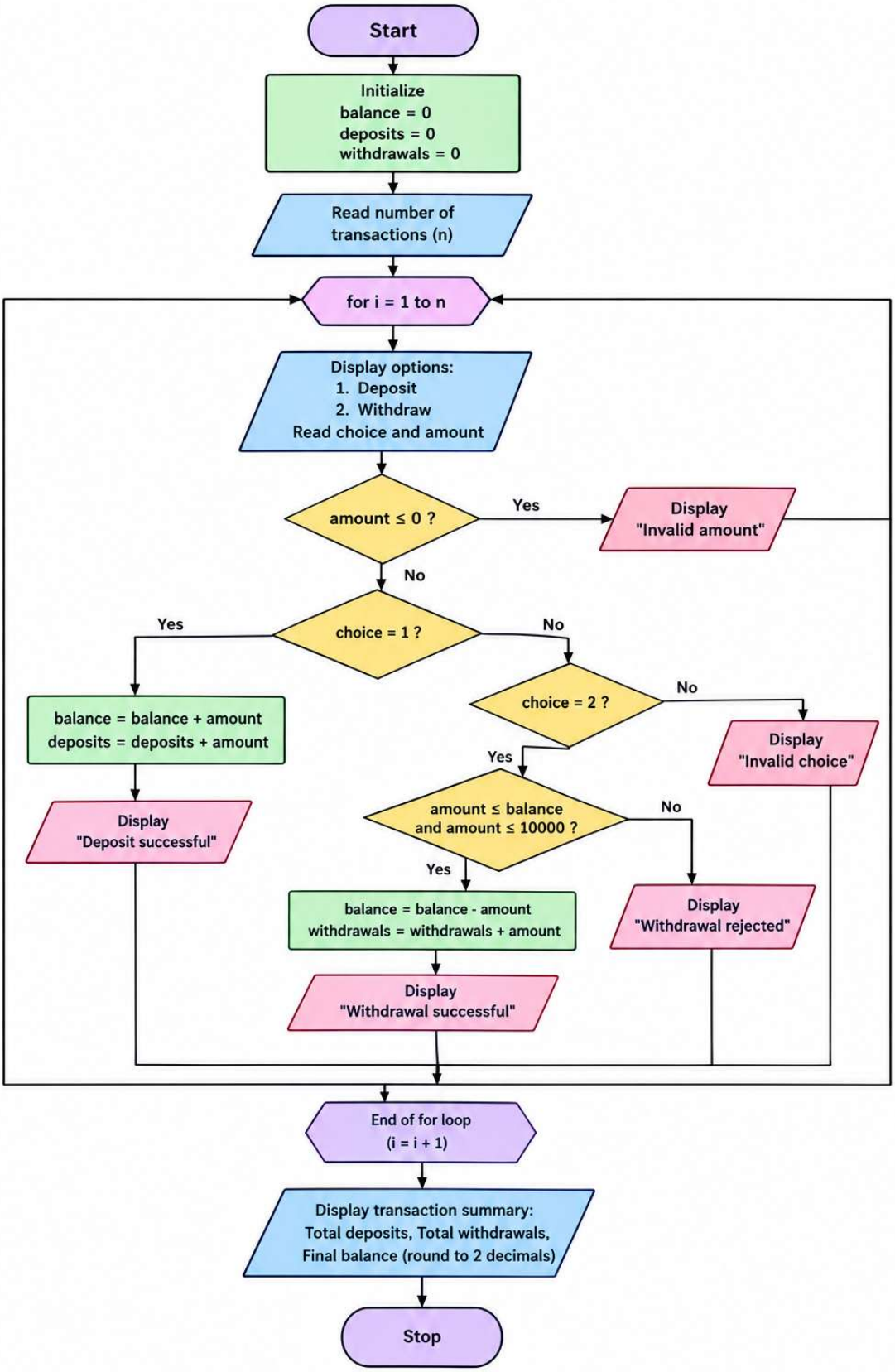
3. Assumptions

- Initial balance is ₹10,000.
- Deposit amount must be positive.
- Withdrawal amount must be positive.
- Maximum withdrawal per transaction is ₹10,000.
- Withdrawal cannot exceed available balance.

4. Algorithm

1. Start.
2. Initialize balance, deposits and withdrawals.
3. Read the number of transactions.
4. Display deposit and withdrawal options.
5. Read the user's choice and amount.
6. Validate the amount.
7. Update balance based on the transaction.
8. Repeat until all transactions are completed.
9. Display transaction summary.
10. Stop.

5. Flowchart


Start
Initialize
balance = 0
deposits = 0
withdrawals = 0
Read number of
transactions (n)
for i = 1 to n
Display options:
1. Deposit
2. Withdraw
Read choice and amount
amount ≤ 0 ?
Yes
Display
"Invalid amount"
No
choice = 1 ?
Yes
No
balance = balance + amount
deposits = deposits + amount
choice = 2 ?
No
Display
"Invalid choice"
Yes
Display
"Deposit successful"
amount ≤ balance
and amount ≤ 10000 ?
No
Yes
balance = balance - amount
withdrawals = withdrawals + amount
Display
"Withdrawal rejected"
Display
"Withdrawal successful"
End of for loop
(i = i + 1)
Display transaction summary:
Total deposits, Total withdrawals,
Final balance (round to 2 decimals)
Stop

6. Pseudocode

```
START
balance = 10000
deposits = 0
withdrawals = 0

INPUT n

FOR each transaction
  INPUT choice, amount

  IF amount <= 0
    PRINT Invalid amount
  ELSE IF choice == 1
    balance = balance + amount
    deposits = deposits + amount
  ELSE IF choice == 2
    IF amount <= balance AND amount <= 10000
      balance = balance - amount
      withdrawals = withdrawals + amount
    ELSE
      PRINT Withdrawal rejected
    END IF
  ELSE
    PRINT Invalid choice
  END IF
END FOR
```

PRINT deposits, withdrawals, balance

STOP

7. Python Program

```
balance = 10000
deposits = 0
withdrawals = 0

n = int(input("Enter number of transactions: "))

for i in range(n):
  print("\n1. Deposit")
  print("2. Withdraw")

  choice = int(input("Enter choice: "))
  amount = float(input("Enter amount: "))

  if amount <= 0:
    print("Invalid amount")

  elif choice == 1:
    balance += amount
    deposits += amount
    print("Deposit successful")

  elif choice == 2:
    if amount <= balance and amount <= 10000:
      balance -= amount
      withdrawals += amount
```

```
        print("Withdrawal successful")
    else:
        print("Withdrawal rejected")

else:
    print("Invalid choice")

print("\n--- Transaction Summary ---")
print("Total deposits:", deposits)
print("Total withdrawals:", withdrawals)
print("Final balance:", round(balance, 2))
```

8. Test Cases

Transaction	Amount	Expected balance
Deposit	₹5,000	₹15,000
Withdraw	₹3,000	₹12,000
Withdraw	₹15,000	Rejected
Deposit	-₹500	Invalid amount

9. Conclusion

This program demonstrates loops, conditional statements and variable updates to manage basic banking transactions.

CASE STUDY 6: Food Wastage Management

1. Problem Statement

Develop a Python program to calculate leftover meals and food wastage percentage for multiple days. Display a warning when food wastage exceeds 20%.

2. Objectives

- Record daily meals prepared and served.

- Calculate leftover meals.
- Calculate wastage percentage.
- Identify high food wastage.

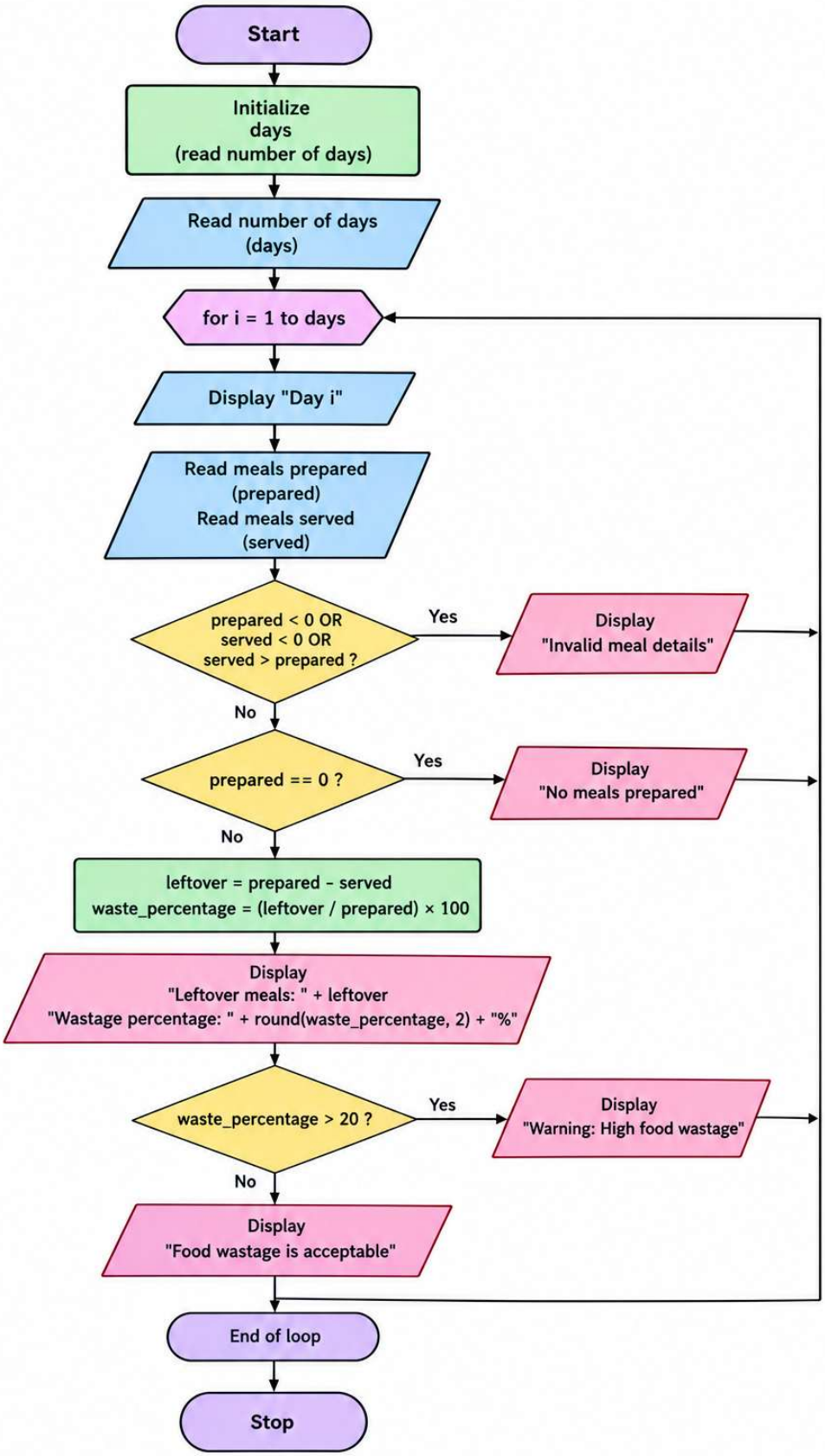
3. Assumptions

- Prepared meals must be non-negative.
- Served meals must be non-negative.
- Served meals cannot exceed prepared meals.
- Wastage above 20% is considered high.

4. Algorithm

1. Start.
2. Read the number of days.
3. Repeat for every day.
4. Input meals prepared and served.
5. Validate the details.
6. Calculate leftover meals.
7. Calculate wastage percentage.
8. Display a warning if wastage exceeds 20%.
9. Stop.

5. Flowchart


Start
Initialize
days
(read number of days)
Read number of days
(days)
for i = 1 to days
Display "Day i"
Read meals prepared
(prepared)
Read meals served
(served)
prepared < 0 OR
served < 0 OR
served > prepared ?
Yes
Display
"Invalid meal details"
No
prepared == 0 ?
Yes
Display
"No meals prepared"
No
leftover = prepared - served
waste_percentage = (leftover / prepared) × 100
Display
"Leftover meals: " + leftover
"Wastage percentage: " + round(waste_percentage, 2) + "%"
waste_percentage > 20 ?
Yes
Display
"Warning: High food wastage"
No
Display
"Food wastage is acceptable"
End of loop
Stop

6. Pseudocode

```
START
INPUT days

FOR each day
  INPUT prepared, served

  IF prepared < 0 OR served < 0 OR served > prepared
    PRINT Invalid details
  ELSE IF prepared == 0
    PRINT No meals prepared
  ELSE
    leftover = prepared - served
    waste = leftover / prepared * 100

    IF waste > 20
      PRINT High food wastage
    ELSE
      PRINT Acceptable wastage
    END IF
  END IF
END FOR

STOP
```

7. Python Program

```
days = int(input("Enter number of days: "))

for i in range(days):
```

```
print("\nDay", i + 1)

prepared = int(input("Enter meals prepared: "))
served = int(input("Enter meals served: "))

if prepared < 0 or served < 0 or served > prepared:
    print("Invalid meal details")

elif prepared == 0:
    print("No meals prepared")

else:
    leftover = prepared - served
    waste_percentage = leftover / prepared * 100

    print("Leftover meals:", leftover)
    print("Wastage percentage:",
          round(waste_percentage, 2), "%")

    if waste_percentage > 20:
        print("Warning: High food wastage")
    else:
        print("Food wastage is acceptable")
```

8. Test Cases

Prepared	Served	Leftover	Wastage
100	90	10	10%
100	70	30	30%

200	150	50	25%
0	0	—	No meals prepared

9. Conclusion

This program helps monitor food wastage and encourages better food management using loops and conditional statements.

CASE STUDY 7: Student Marks Analysis

1. Problem Statement

Develop a Python program to collect marks of multiple students in multiple subjects. Calculate student-wise averages, subject-wise averages and identify the top-performing student.

2. Objectives

- Store student marks using lists.
- Calculate student averages.
- Calculate subject averages.
- Identify the highest-performing student.

3. Assumptions

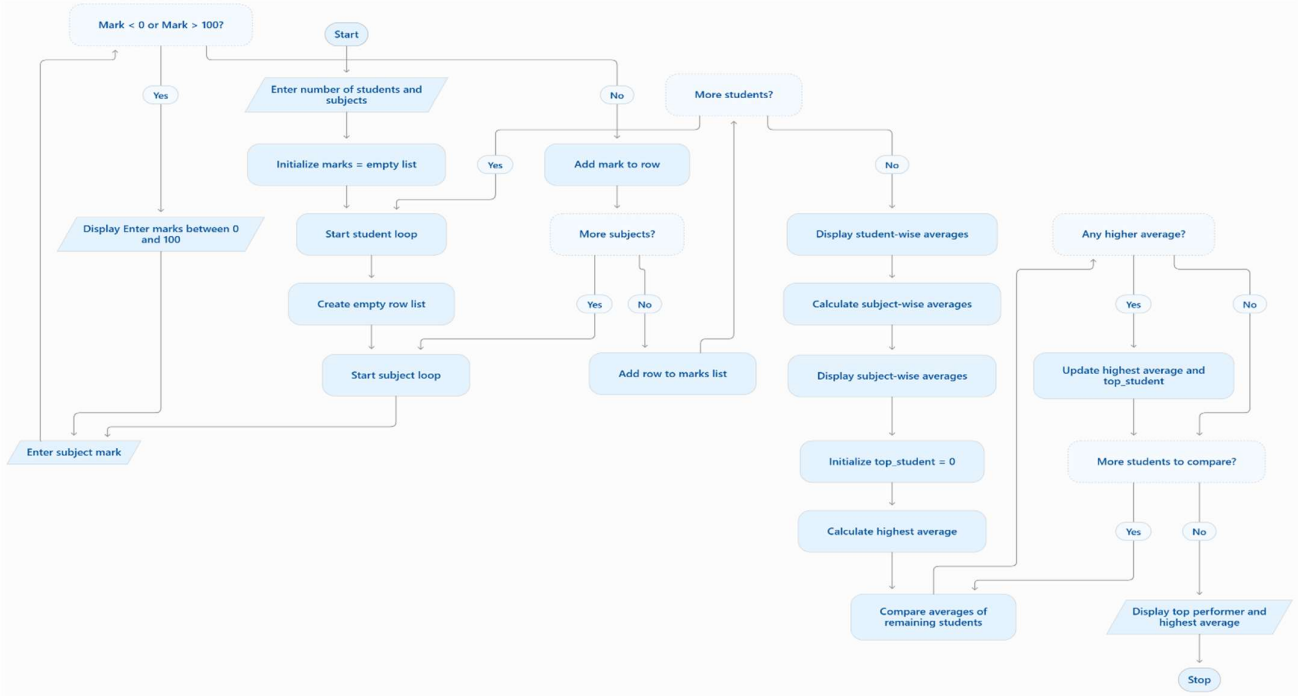
- Number of students and subjects must be positive.
- Marks must be between 0 and 100.
- All students have the same number of subjects.

4. Algorithm

1. Start.
2. Input the number of students and subjects.
3. Create an empty marks list.
4. Read marks for every student.
5. Validate each mark.
6. Calculate student-wise averages.
7. Calculate subject-wise averages.

8. Find the highest average.
9. Display the top performer.
10. Stop.

5. Flowchart



6. Pseudocode

START

INPUT students, subjects

IF students <= 0 OR subjects <= 0

PRINT Invalid dimensions

ELSE

CREATE marks list

FOR each student

FOR each subject

```
        INPUT mark
        VALIDATE mark between 0 and 100
        STORE mark
      END FOR
    END FOR

    CALCULATE student averages
    CALCULATE subject averages
    FIND highest average
    PRINT top performer
END IF

STOP
```

7. Python Program

```
students = int(input("Enter number of students: "))
subjects = int(input("Enter number of subjects: "))

if students <= 0 or subjects <= 0:
    print("Invalid dimensions")

else:
    marks = []

    for i in range(students):
        row = []
        print("\nStudent", i + 1)
```

```
    for j in range(subjects):
        mark = float(input("Enter mark: "))

        while mark < 0 or mark > 100:
            print("Enter marks between 0 and 100")
            mark = float(input("Enter mark again: "))

        row.append(mark)

    marks.append(row)

print("\nStudent-wise averages:")

for i in range(students):
    average = sum(marks[i]) / subjects
    print("Student", i + 1, ":", round(average, 2))

print("\nSubject-wise averages:")

for j in range(subjects):
    total = 0

    for i in range(students):
        total += marks[i][j]

    average = total / students
    print("Subject", j + 1, ":", round(average, 2))
```

```
top_student = 0
highest_average = sum(marks[0]) / subjects

for i in range(1, students):
    average = sum(marks[i]) / subjects

    if average > highest_average:
        highest_average = average
        top_student = i

print("\nTop performer: Student", top_student + 1)
print("Highest average:", round(highest_average, 2))
```

8. Test Cases

Consider three students and two subjects.

Student	Subject 1	Subject 2	Average
Student 1	80	90	85
Student 2	70	80	75
Student 3	90	95	92.5

- Subject 1 average: 80
- Subject 2 average: 88.33
- Top performer: Student 3

9. Conclusion

This program demonstrates lists, nested loops and mathematical calculations to analyse student academic performance.

CASE STUDY 8: Number Diamond Pattern

1. Problem Statement

Develop a Python program to print a symmetrical number diamond pattern using nested loops. The number of rows is provided by the user.

2. Objectives

- Understand nested loops.
- Print number patterns.
- Use spaces for alignment.
- Develop logical thinking.

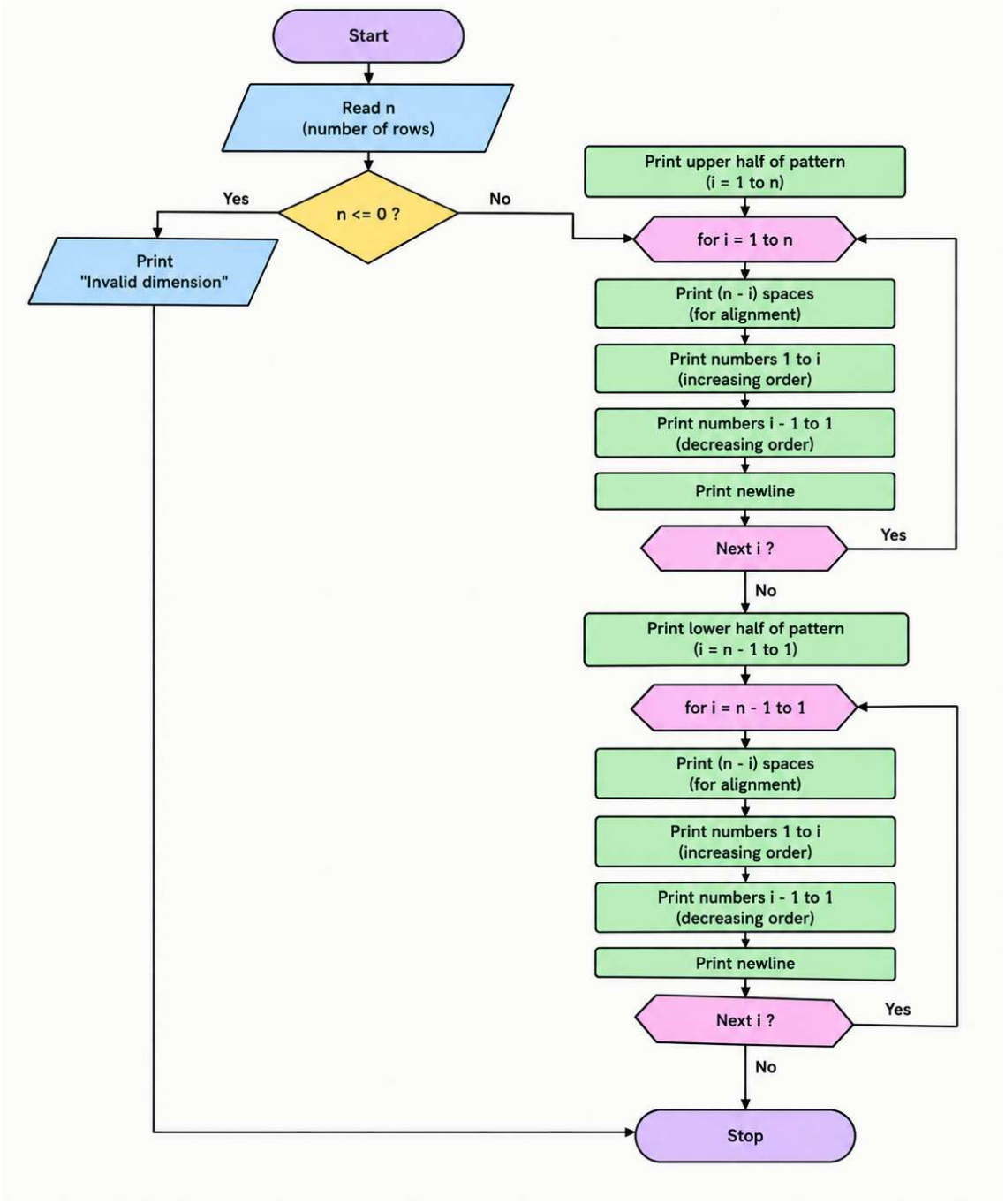
3. Assumptions

- The number of rows must be positive.
- The pattern must be symmetrical.
- Numbers increase and then decrease in every row.

4. Algorithm

1. Start.
2. Read the number of rows.
3. Check whether the number is positive.
4. Print the upper half of the diamond.
5. Print the lower half of the diamond.
6. Display the pattern.
7. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n

IF n <= 0
  PRINT Invalid dimension
ELSE
  FOR i = 1 TO n
    PRINT spaces
    PRINT increasing numbers
    PRINT decreasing numbers
  END FOR

  FOR i = n-1 DOWNTO 1
    PRINT spaces
    PRINT increasing numbers
    PRINT decreasing numbers
  END FOR
END IF

STOP
```

7.Python Program

```
n = int(input("Enter the number of rows: "))

if n <= 0:
  print("Invalid dimension")

else:
  for i in range(1, n + 1):
```

```
    print(" " * (n - i), end="")

    for j in range(1, i + 1):
        print(j, end="")

    for j in range(i - 1, 0, -1):
        print(j, end="")

    print()

for i in range(n - 1, 0, -1):
    print(" " * (n - i), end="")

    for j in range(1, i + 1):
        print(j, end="")

    for j in range(i - 1, 0, -1):
        print(j, end="")

    print()
```

8. Test Cases

Input: 4

Expected output:

```
   1
  121
 12321
1234321
 12321
```

121

1

Input: 0

Expected output:

Invalid dimension

9. Conclusion

This program demonstrates nested loops and pattern printing using Python. It improves understanding of loop execution and number sequences

CASE STUDY 9: Password Validation System

1. Problem Statement

Develop a Python program to validate a password based on security rules. The password must contain uppercase letters, lowercase letters, digits and special characters.

2. Objectives

- Validate password length.
- Check uppercase and lowercase letters.
- Check digits and special characters.
- Display all password errors.

3. Assumptions

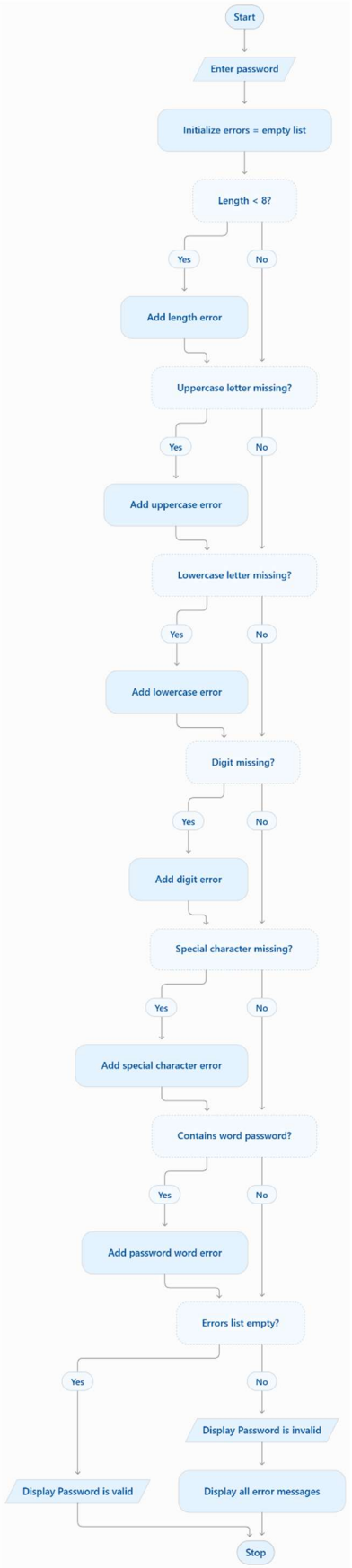
- Minimum password length is 8 characters.
- At least one uppercase letter is required.
- At least one lowercase letter is required.
- At least one digit is required.
- At least one special character is required.
- The word "password" must not be included, regardless of case.

4. Algorithm

1. Start.

2. Read the password.
3. Create an empty error list.
4. Check password length.
5. Check uppercase and lowercase letters.
6. Check for digits and special characters.
7. Check whether the word password is present.
8. If errors exist, display invalid password and errors.
9. Otherwise, display valid password.
10. Stop.

5. Flowchart


Start
Enter password
Initialize errors = empty list
Length < 8?
Yes
No
Add length error
Uppercase letter missing?
Yes
No
Add uppercase error
Lowercase letter missing?
Yes
No
Add lowercase error
Digit missing?
Yes
No
Add digit error
Special character missing?
Yes
No
Add special character error
Contains word password?
Yes
No
Add password word error
Errors list empty?
Yes
No
Display Password is invalid
Display Password is valid
Display all error messages
Stop

6. Pseudocode

```
START
INPUT password
errors = empty list

IF length < 8
   ADD length error
END IF

CHECK uppercase
CHECK lowercase
CHECK digit
CHECK special character
CHECK forbidden word

IF errors exist
   PRINT Invalid password
   PRINT errors
ELSE
   PRINT Valid password
END IF

STOP
```

7.Python Program

```
password = input("Enter a password: ")

errors = []
```

```
if len(password) < 8:
    errors.append("Password must contain at least 8 characters")

if not any(ch.isupper() for ch in password):
    errors.append("Include at least one uppercase letter")

if not any(ch.islower() for ch in password):
    errors.append("Include at least one lowercase letter")

if not any(ch.isdigit() for ch in password):
    errors.append("Include at least one digit")

if not any(not ch.isalnum() for ch in password):
    errors.append("Include at least one special character")

if "password" in password.lower():
    errors.append("Do not use the word 'password'")

if errors:
    print("Password is invalid")

    for error in errors:
        print("-", error)

else:
    print("Password is valid")
```

8. Test Cases

Input	Expected result
Abc@1234	Valid password
abc123	Invalid – multiple errors
ABCDEFGH1@	Invalid – lowercase missing
Abcpassword1@	Invalid – forbidden word
Abcdef12	Invalid – special character missing

9. Conclusion

This program demonstrates string handling, list operations and conditional checking to create a basic password validation system.

CASE STUDY 10: Student Performance Analysis

1. Problem Statement

Develop a Python program to analyse student performance over three examinations. Calculate average marks, identify improvement or decline and classify student performance.

2. Objectives

- Collect examination marks.
- Calculate average marks.
- Compare first and last examination marks.
- Identify performance trends.
- Classify students based on averages.

3. Assumptions

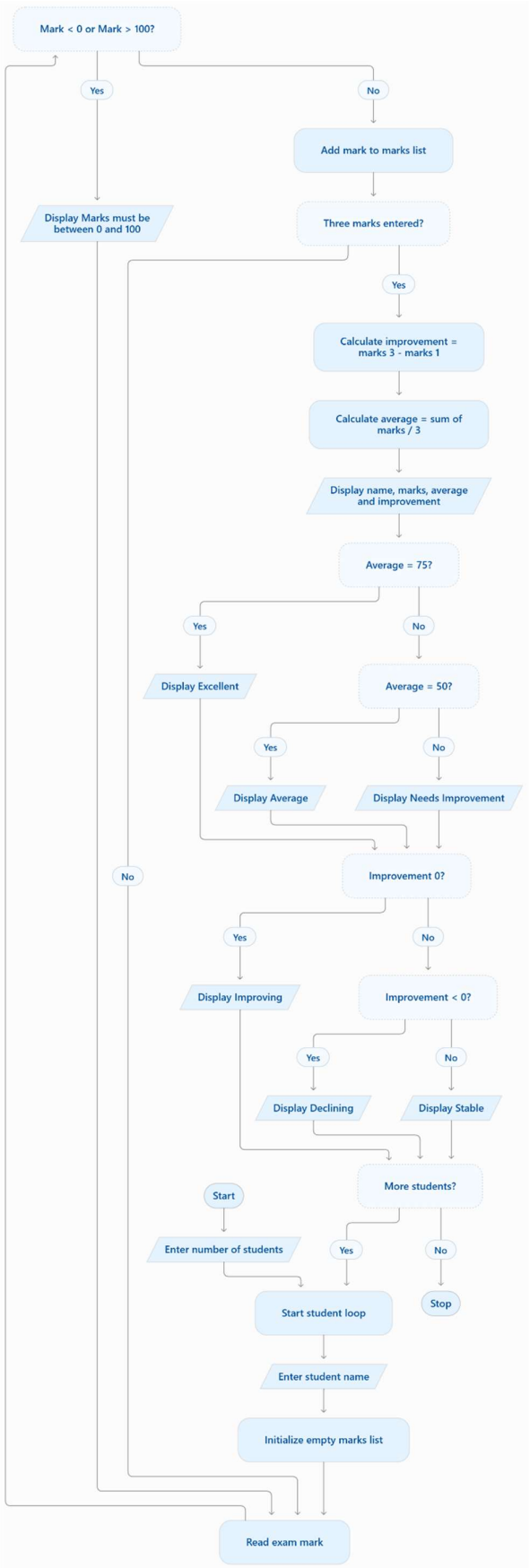
- Each student has three examination marks.
- Marks must be between 0 and 100.
- Average of 75 or above: Excellent.
- Average of 50–74.99: Average.
- Below 50: Needs Improvement.

- Improvement is calculated by subtracting the first exam mark from the last exam mark.

4. Algorithm

1. Start.
2. Input the number of students.
3. Repeat for every student.
4. Read student name.
5. Read three examination marks.
6. Validate the marks.
7. Calculate average marks.
8. Calculate performance improvement.
9. Classify the student.
10. Identify performance trend.
11. Display results.
12. Stop.

5. Flowchart


Mark < 0 or Mark > 100?
Yes
No
Add mark to marks list
Display Marks must be between 0 and 100
Three marks entered?
Yes
Calculate improvement = marks 3 - marks 1
Calculate average = sum of marks / 3
Display name, marks, average and improvement
Average = 75?
Yes
No
Display Excellent
Average = 50?
Yes
No
Display Average
Display Needs Improvement
No
Improvement 0?
Yes
No
Display Improving
Improvement < 0?
Yes
No
Display Declining
Display Stable
More students?
Start
Enter number of students
Yes
No
Stop
Start student loop
Enter student name
Initialize empty marks list
Read exam mark

6. Pseudocode

```
START
INPUT n

FOR each student
  INPUT name
  INPUT three exam marks
  VALIDATE marks

  average = sum(marks) / 3
  improvement = third mark - first mark

  IF average >= 75
    classification = Excellent
  ELSE IF average >= 50
    classification = Average
  ELSE
    classification = Needs Improvement
  END IF

  IF improvement > 0
    trend = Improving
  ELSE IF improvement < 0
    trend = Declining
  ELSE
    trend = Stable
  END IF
```

```
  PRINT student report
END FOR

STOP
```

7.Python Program

```
n = int(input("Enter number of students: "))

for i in range(n):
  name = input("\nEnter student name: ")
  marks = []

  for j in range(3):
    mark = float(input("Enter exam mark: "))

    while mark < 0 or mark > 100:
      print("Marks must be between 0 and 100")
      mark = float(input("Enter mark again: "))

    marks.append(mark)

  improvement = marks[2] - marks[0]
  average = sum(marks) / 3

  print("\nStudent:", name)
  print("Exam marks:", marks)
  print("Average:", round(average, 2))
  print("Change from first to last exam:", improvement)
```

```
    if average >= 75:
        print("Classification: Excellent")
    elif average >= 50:
        print("Classification: Average")
    else:
        print("Classification: Needs Improvement")

    if improvement > 0:
        print("Performance trend: Improving")
    elif improvement < 0:
        print("Performance trend: Declining")
    else:
        print("Performance trend: Stable")
```

8. Test Cases

Student	**Exam 1**	**Exam 2**	**Exam 3**	**Average**	**Classification**
Ravi	80	85	90	85	Excellent
Sita	60	65	70	65	Average
Ram	40	45	50	45	Needs Improvement

Expected performance trends:

- Ravi: Improving
- Sita: Improving
- Ram: Improving

9. Conclusion

This program demonstrates lists, loops, arithmetic operations and conditional statements to analyse student academic performance.